Master Theorem and Linear Time Sorts

1 Master Theorem, one Theorem to bound them all

The master theorem is the result of the recursive expansion. It handles recursive functions at the form of $T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$, for a positive function $f : \mathbb{N} \to \mathbb{R}^+$.

Master Theorem.
First, Consider the case that $f = \Theta(n^c)$. Let $a \geq 1, b > 1$ and $c \geq 0$. Then:

1. If $\frac{a}{b^c} < 1$ then $T(n) = \Theta(n^c)$ **(f wins)**.
2. If $\frac{a}{b^c} = 1$ then $T(n) = \Theta(n^c \log_b(n))$.
3. If $\frac{a}{b^c} > 1$ then $T(n) = \Theta\left(n^{\log_b(a)}\right)$ **(f loses)**.

Example 1.

1. $T(n) = 4T(\frac{n}{2}) + d \cdot n \Rightarrow T(n) = \Theta(n^2)$ according to case (3).
2. $T(n) = 3T(\frac{n}{2}) + d \cdot n \Rightarrow T(n) = \Theta(n^{\log_2(3)})$ also due to case (3).

Master Theorem, General Version.
Now, let's generalize the simple version for an arbitrary positive f and let $a \geq 1$, $b > 1$.

1. If there exist $\varepsilon > 0, c < 1$, and $n_0 \in \mathbb{N}$ such that $f(n) = \Omega\left(n^{\log_b(a)+\varepsilon}\right)$ and for every $n > n_0$ it holds that $a \cdot f\left(\frac{n}{b}\right) \leq cf(n)$, then $T(n) = \Theta(f(n))$ **(f wins)**.
2. If $f(n) = O\left(n^{\log_b(a)-\varepsilon}\right)$ for some $\varepsilon > 0$ then $T(n) = \Theta\left(n^{\log_b(a)}\right)$ **(f loses)**.
3. If $f(n) = \Theta\left(n^{\log_b(a)}\right)$ then $T(n) = \Theta\left(n^{\log_b(a)} \log(n)\right)$

Example 2. Solve the following recursive relations:

1. $T(n) = T(\frac{2n}{3}) + 1$.

   So, $f(n) = 1 = \Theta(n^{\log_{\frac{3}{2}}(1)})$, which matches the second case, hence:

   $$T(n) = \Theta(n^{\log_{\frac{3}{2}}(1)} \log n) \to \Theta(\log n)$$

   .

2. $T(n) = 3T\left(\frac{n}{4}\right) + n \log n$.

   Clearly, $f(n) = \Omega(n^{\log_4(3)+\varepsilon})$, which suggests that the first case might hold. To ensure that, we have to show the existence of constants c, n_0 that satisfy the second condition of the case. Pick $c = \frac{3}{4}$, and using the monotony of the log-function, we have:

   $$af(\frac{n}{b}) \ = \frac{3n}{4} \log(\frac{n}{4}) = \frac{3}{4} n \log(\frac{n}{4}) \leq \frac{3}{4} n \log(n)$$

   Thus, $T(n) = \Theta(f(n)) = \Theta(n \log n)$.

3. $T(n) = 3T(n^{\frac{1}{3}}) + \log \log n$.

   Let $m = \log n$, thus:

   $$\Rightarrow T(n) = T(2^m) = 3T(2^{\frac{m}{3}}) + \log m$$

   And denote $S = S(m) = T(2^m)$, so we get the recursive form:

   $$\to S(m) = 3T(2^{\frac{m}{3}}) + \log m = 3S(\frac{m}{3}) + \log m$$

   Thus, we can apply the master theorem according to the third case:

   $$\log m = O(m^{\log_3(3)-\varepsilon})$$
   $$\to T(n) = T(2^m) = S(m) = \Theta(m) = \Theta(\log(n))$$

2 Linear Time Sorts (Assumptions over the inputs)

You will see in the next lecture that any comparison sort, namely a sorting algorithm that assumes nothing about the given input except that its items can be compared, has an $\Omega(n \log n)$ lower bound on its running time. We will now see how adding assumptions about the input structure might help us bypass the limit and achieve an asymptotically better running time.

Example 3. Assume that the input is permutation over $\{1, 2, 3, .., n\}$. For example:

$$A = 5, 1, 2, 3, 6, 4, 7, 8$$

Write an algorithm that sorts given the assumption. What is its running time?

Solution. Just return the sequence $1, 2, .., n$. Requires $\Theta(n)$ to print the numbers.

Example 4. Assume that the input is an array containing only 0s and 1s. For example:

$$A = 0, 1, 1, 1, 0, 0, 1, 0, 1$$

Write an algorithm that sorts given the assumption. What is its running time?

Solution. First, count the zeros. Let x be the amount. Then print 0 for x times and print 1 for $n - x$ times. This can be done in $\Theta(n)$ time.

Counting sort. Counting sort assumes that each of the n input elements is an **integer** which is at most k. It runs in $\Theta(n+k)$ time, so when $k = O(n)$, counting sort runs in $\Theta(n)$ time. It first determines, for each input element x, the number of elements less than or equal to x. Then, it uses this information to place element x directly into its position in the output array. For example, if there are 17 elements less than or equal to x, then x will be placed in position 17. However, we need to make some adjustments to this method to handle cases where multiple elements have the **same value**. We do not want all of them to end up in the same position.

Notice that the Counting sort can beat the lower bound of $\Omega(n \log n)$ only because it is not a comparison sort. In fact, no comparisons between input elements occur anywhere in the code. Instead, counting sort uses the actual values of the elements to 'index' them into an array.

An important property of the counting sort is that it is **stable**.

> **Stable Sort.** We will say that a sorting algorithm is stable if elements with the same value appear in the output array in the same order as they do in the input array.

Counting sort's stability is important for another reason: counting sort is often used as a subroutine in radix sort. As we shall see in the next section, in order for radix sort to work correctly, counting sort must be stable.

Algorithm 1: Counting-sort(A, n, k)

```
let B and C be new arrays at size n and k
for i ∈ [1, k] do
    C_i ← 0
end
for j ← [1, n] do
    C_{A_j} ← C_{A_j} + 1
end
for i ∈ [2, k] do
    C_i ← C_i + C_{i−1}
end
for j ∈ [n, 1] do
    B_{C_{A_j}} ← A_j
    C_{A_j} ← C_{Aj}−1   // to handle duplicate values
end
return B
```

Example 5. Consider the array $A = \{3, 1, 3, 2, 1\}$. Denote by $C^{(i)}$ the array C at the end of the ith iteration of the second loop, where $C^{(0)}$ is the array prior to the execution of the loop.

$$C^{(0)} = \{0, 0, 0, 0, 0\}$$
$$C^{(1)} = \{0, 0, 1, 0, 0\}$$
$$C^{(2)} = \{1, 0, 1, 0, 0\}$$
$$C^{(3)} = \{1, 0, 2, 0, 0\}$$
$$C^{(4)} = \{1, 1, 2, 0, 0\}$$
$$C^{(5)} = \{2, 1, 2, 0, 0\}$$

Then, after the accumulation in the third loop, C becomes:

$$C = \{2, 3, 5, 0, 0\}$$

Now, let $B^{(i)}, C^{(5+i)}$ be the arrays B and C at the end of the ith iteration of the fourth loop.

$$B^{(1)} = \{0, 1, 0, 0, 0\} \quad C^{(6)} = \{1, 3, 5, 0, 0\}$$
$$B^{(2)} = \{0, 1, 2, 0, 0\} \quad C^{(7)} = \{1, 2, 5, 0, 0\}$$
$$B^{(3)} = \{0, 1, 2, 0, 3\} \quad C^{(8)} = \{1, 2, 4, 0, 0\}$$
$$B^{(4)} = \{1, 1, 2, 0, 3\} \quad C^{(9)} = \{0, 2, 4, 0, 0\}$$
$$B^{(5)} = \{1, 1, 2, 3, 3\} \quad C^{(9)} = \{0, 2, 3, 0, 0\}$$

Observes that since the order in the fourth loop is from the big to the small (or right to left), it's guaranteed that between two elements with equal value in A, the rightmost among the two will be placed at a higher index in B.

Radix sort The Radix-sort procedure assumes that each element in the array A has d digits, where digit 1 is the lowest-order digit and digit d is the highest-order digit.

Algorithm 2: radix-sort(A, n, d)

```
for i ∈ [1, d] do
    use a stable sort to sort array A on digit i
end
```

Correctness Proof. By induction on the column being sorted.

- Base. Where $d = 1$, the correctness follows immediately from the correctness of our base sort subroutine.
- Induction Assumption. Assume that Radix-sort is correct for any array of numbers containing at most $d - 1$ digits.
- Step. Let A' be the algorithm output. Consider $x, y \in A$. Assume without losing generality that $x > y$. Denote by x_d, y_d their d-digit and by $x_{/d}, y_{/d}$ the numbers obtained by taking only the first $d - 1$ digits of x, y. Separate in two cases:
  - If $x_d > y_d$ then a scenario in which x appear prior to y is imply contradiction to the correctness of our subroutine.
  - So consider the case in which $x_d = y_d$. In that case, it must hold that $x_{/d} > y_{/d}$. Then the appearance of x prior to y either contradicts the assumption that the base algorithm we have used is stable or that x appears before y at the end of the $d - 1$ iteration. Which contradicts the induction assumption.

The analysis of the running time depends on the stable sort used as the intermediate sorting algorithm. When each digit lies in the range 0 to k–1 (so that it can take on k possible values), and k is not too large, counting sort is the obvious choice. Each pass over n d-digit numbers then takes $\Theta(n + k)$ time. There are d passes, and so the total time for radix sort is $\Theta\left(d(n + k)\right)$.

Bucket sort. Bucket sort divides the interval [0, 1) into n equal-sized subintervals, or buckets, and then distributes the n input numbers into the buckets. Since the inputs are uniformly and independently distributed over [0, 1), we do not expect many numbers to fall into each bucket. To produce the output, we simply sort the numbers in each bucket and then go through the buckets in order, listing the elements in each.

Note 2: We will return to analyze the expected (average) running time after the lecture on probability.

Algorithm 3: bucket-sort(A, n)

```
let B[0 : n − 1] be a new array
for i ← [0, n−1] do
    make B_i an empty list
end
for i ← [1, n] do
    insert A_i into list B_⌊nA_i⌋]
end
for i ← [0, n−1] do
    sort list B_i
end
concatenate the lists B_0, B_1, .., B_{n−1} together and
return the concatenated lists
```